

Q1. Explain Features of Java, JVM Architecture, Data Types and Command Line Arguments

(A) FEATURES OF JAVA

Definition

Java is a high-level, object-oriented, platform-independent programming language developed by Sun Microsystems.

Features of Java

1. Simple

- Easy syntax
- No pointers
- Automatic memory management

2. Object-Oriented

Supports:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

3. Platform Independent

Java follows:

Write Once Run Anywhere (WORA)

Flow:

```
.java → javac → bytecode(.class) → JVM
```

4. Secure

- Bytecode verification
- No direct memory access

- Exception handling

5. Robust

- Strong memory management
- Garbage collection
- Exception handling

6. Portable

Programs run on all operating systems.

7. Multithreaded

Supports multiple threads simultaneously.

8. Distributed

Supports network programming.

9. Dynamic

Classes loaded during runtime.

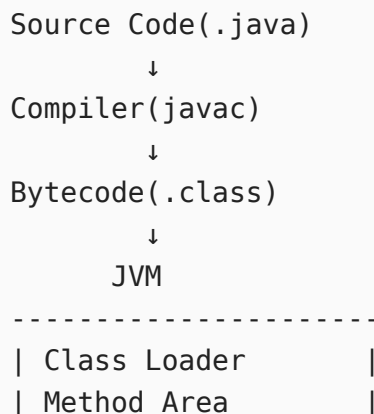
(B) JVM ARCHITECTURE

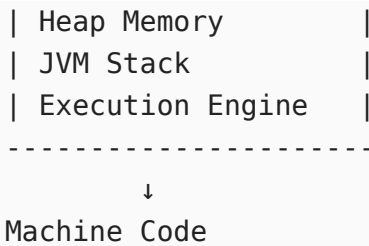
JVM

JVM stands for Java Virtual Machine.

It executes Java bytecode.

JVM Architecture Diagram





Components

1. Class Loader

Loads class files into memory.

2. Method Area

Stores class-level data.

3. Heap Memory

Stores objects.

4. Stack Memory

Stores method calls and local variables.

5. Execution Engine

Executes bytecode.

(C) DATA TYPES

Primitive Data Types

Data Type	Size
byte	1 byte
short	2 bytes
int	4 bytes
long	8 bytes
float	4 bytes
double	8 bytes

Data Type	Size
char	2 bytes
boolean	1 bit

Example

```
class Demo {  
    public static void main(String args[]) {  
  
        int a = 10;  
        double d = 12.5;  
        char ch = 'A';  
  
        System.out.println(a);  
        System.out.println(d);  
        System.out.println(ch);  
    }  
}
```

(D) COMMAND LINE ARGUMENTS

Definition

Arguments passed during execution are called command line arguments.

Example

```
class Test {  
  
    public static void main(String args[]) {  
  
        System.out.println(args[0]);  
        System.out.println(args[1]);  
    }  
}
```

Execution

```
java Test Robby 20
```

Output

```
Robby  
20
```

Q2. Explain OOP Concepts with Programs

OBJECT ORIENTED PROGRAMMING

OOP is a programming approach based on objects and classes.

Four Pillars of OOP

1. Encapsulation
 2. Inheritance
 3. Polymorphism
 4. Abstraction
-

(A) ENCAPSULATION

Definition

Wrapping data and methods together into one unit.

Example

```
class Bank {  
  
    private int balance = 1000;  
  
    public void deposit(int amt) {  
        balance += amt;  
    }  
  
    public void display() {  
        System.out.println(balance);  
    }  
}
```

```
}  
}
```

Advantages

- Data security
 - Data hiding
 - Controlled access
-

(B) INHERITANCE

Definition

Acquiring properties of parent class.

Example

```
class Animal {  
  
    void eat() {  
        System.out.println("Eating");  
    }  
}  
  
class Dog extends Animal {  
  
    void bark() {  
        System.out.println("Barking");  
    }  
}  
  
class Demo {  
  
    public static void main(String args[]) {  
  
        Dog d = new Dog();  
  
        d.eat();  
        d.bark();  
    }  
}
```

Types

- Single
 - Multilevel
 - Hierarchical
-

(C) POLYMORPHISM

Definition

One thing having many forms.

Types

1. Compile-time polymorphism
 2. Runtime polymorphism
-

Method Overloading

```
class Add {  
  
    int sum(int a,int b) {  
        return a+b;  
    }  
  
    int sum(int a,int b,int c) {  
        return a+b+c;  
    }  
}
```

Method Overriding

```
class Animal {  
  
    void sound() {  
        System.out.println("Animal Sound");  
    }  
}
```

```
}  
  
class Dog extends Animal {  
  
    void sound() {  
        System.out.println("Dog Barks");  
    }  
}
```

(D) ABSTRACTION

Definition

Hiding implementation details.

Example

```
abstract class Shape {  
  
    abstract void draw();  
}  
  
class Circle extends Shape {  
  
    void draw() {  
        System.out.println("Drawing Circle");  
    }  
}
```

Q3. Explain Constructors, this, super and static Keywords

(A) CONSTRUCTORS

Definition

Special method used to initialize objects.

Types

1. Default constructor
 2. Parameterized constructor
-

Default Constructor

```
class Student {  
  
    Student() {  
        System.out.println("Default Constructor");  
    }  
}
```

Parameterized Constructor

```
class Student {  
  
    int id;  
  
    Student(int id) {  
        this.id = id;  
    }  
}
```

(B) this KEYWORD

Uses

- Refers current object
- Calls current constructor

Example

```
class Demo {
```

```
int x;

Demo(int x) {
    this.x = x;
}
}
```

(C) super KEYWORD

Uses

- Access parent variable
- Access parent method
- Call parent constructor

Example

```
class Animal {

    String color = "White";
}

class Dog extends Animal {

    String color = "Black";

    void show() {
        System.out.println(super.color);
    }
}
```

(D) static KEYWORD

Definition

Belongs to class, not object.

Static Variable Example

```
class Student {  
  
    static String college = "CUJ";  
}
```

Static Method Example

```
class Demo {  
  
    static void display() {  
        System.out.println("Hello");  
    }  
}
```

Q4. Explain Exception Handling in Java

EXCEPTION HANDLING

Definition

Process of handling runtime errors.

try-catch Example

```
class Demo {  
  
    public static void main(String args[]) {  
  
        try {  
  
            int a = 10/0;  
  
        }  
  
        catch(ArithmeticException e) {  
  
            System.out.println("Division by zero not allowed");  
        }  
    }  
}
```

```
    }  
  
    System.out.println("Program Continues");  
}  
}
```

finally Block

Definition

Always executes.

Example

```
try {  
  
    int a = 10/2;  
}  
  
catch(Exception e) {  
  
    System.out.println(e);  
}  
  
finally {  
  
    System.out.println("Finally Executed");  
}
```

throw Keyword

Example

```
class Demo {  
  
    public static void main(String args[]) {  
  
        int age = 15;  
  
        if(age < 18) {
```

```
        throw new ArithmeticException("Invalid Age");
    }
}
}
```

throws Keyword

Example

```
import java.io.*;

class Demo {

    public static void main(String args[])
        throws IOException {

        FileReader f = new FileReader("a.txt");
    }
}
```

User Defined Exception

```
class MyException extends Exception {

    MyException(String msg) {
        super(msg);
    }
}

class Test {

    static void check(int age)
        throws MyException {

        if(age < 18) {
            throw new MyException("Not Eligible");
        }
    }
}
```

```
}  
}
```

Q5. Explain Multithreading in Java

MULTITHREADING

Definition

Executing multiple threads simultaneously.

Advantages

- Faster execution
 - Better CPU utilization
 - Parallel processing
-

THREAD LIFE CYCLE

```
NEW  
↓  
RUNNABLE  
↓  
RUNNING  
↓  
WAITING/BLOCKED  
↓  
TERMINATED
```

Creating Thread using Thread Class

```
class MyThread extends Thread {  
  
    public void run() {
```

```
        System.out.println("Thread Running");
    }
}

class Demo {

    public static void main(String args[]) {

        MyThread t = new MyThread();

        t.start();
    }
}
```

Creating Thread using Runnable Interface

```
class MyThread implements Runnable {

    public void run() {

        System.out.println("Runnable Thread");
    }
}

class Demo {

    public static void main(String args[]) {

        MyThread obj = new MyThread();

        Thread t = new Thread(obj);

        t.start();
    }
}
```

Important Thread Methods

Method	Purpose
start()	Starts thread
sleep()	Pause thread
join()	Wait thread
isAlive()	Check running

Q6. Explain Java I/O Streams

JAVA I/O STREAMS

Definition

Streams are used for input and output operations.

Types

1. Byte Stream
 2. Character Stream
-

FileInputStream

```
import java.io.*;

class Demo {

    public static void main(String args[])
        throws Exception {

        FileInputStream fin =
            new FileInputStream("a.txt");

        int i;

        while((i=fin.read())!=-1) {

            System.out.print((char)i);
```



```
    }  
  
    fin.close();  
}  
}
```

FileOutputStream

```
import java.io.*;  
  
class Demo {  
  
    public static void main(String args[])  
        throws Exception {  
  
        FileOutputStream fout =  
            new FileOutputStream("a.txt");  
  
        String s = "Hello Java";  
  
        fout.write(s.getBytes());  
  
        fout.close();  
    }  
}
```

BufferedReader

```
BufferedReader br =  
    new BufferedReader(  
        new FileReader("a.txt")  
    );
```

BufferedWriter

```
BufferedWriter bw =  
new BufferedWriter(  
new FileWriter("a.txt")  
);
```

File Class

```
File f = new File("a.txt");  
  
System.out.println(f.exists());
```

Q7. Explain Collection Framework

COLLECTION FRAMEWORK

Definition

Framework used to store and manipulate data dynamically.

Interfaces

- List
 - Set
 - Queue
 - Map
-

ARRAYLIST

```
import java.util.*;  
  
class Demo {  
  
    public static void main(String args[]) {
```

```
ArrayList<String> list =  
    new ArrayList<>();  
  
list.add("Java");  
list.add("Python");  
  
System.out.println(list);  
}  
}
```

QUEUE

```
Queue<Integer> q =  
    new LinkedList<>();  
  
q.add(10);  
q.add(20);  
  
System.out.println(q.poll());
```

HASHMAP

```
HashMap<Integer,String> map =  
    new HashMap<>();  
  
map.put(1,"Java");  
map.put(2,"Python");
```

ITERATOR

```
Iterator<String> it =  
    list.iterator();  
  
while(it.hasNext()) {
```

```
System.out.println(it.next());  
}
```

COMPARATOR

```
Collections.sort(list,  
(a,b) -> b.compareTo(a));
```

Used for custom sorting.

Q8. Explain AWT and Event Handling

AWT

AWT stands for Abstract Window Toolkit.

Used for creating GUI applications.

AWT Hierarchy

```
Component  
  ↓  
Container  
  ↓  
-----  
|           |  
Window     Panel  
  ↓  
Frame
```

Components

Component	Purpose
Button	Click action
Label	Display text
TextField	Input text
Checkbox	Selection

FLOWLAYOUT

```
Frame f = new Frame();  
  
f.setLayout(new FlowLayout());
```

GRIDLAYOUT

```
f.setLayout(new GridLayout(2,2));
```

EVENT HANDLING

Definition

Handling user-generated events.

Components

1. Event Source
2. Event Object
3. Event Listener

ActionEvent Program

```
import java.awt.*;
import java.awt.event.*;

class Demo implements ActionListener {

    Frame f;
    Button b;

    Demo() {

        f = new Frame();

        b = new Button("Click");

        b.addActionListener(this);

        f.add(b);

        f.setSize(300,300);

        f.setVisible(true);
    }

    public void actionPerformed(
   (ActionEvent e) {

        System.out.println(
        "Button Clicked");
    }

    public static void main(
    String args[]) {

        new Demo();
    }
}
```

Q9. Explain Regular Expressions

REGULAR EXPRESSIONS

Definition

Regex is used for pattern matching.

Uses

- Validation
- Searching
- Filtering

Important Symbols

Symbol	Meaning
.	Any character
*	Zero or more
+	One or more
?	Optional
[]	Range

Character Classes

Pattern	Meaning
\d	Digit
\D	Non-digit
\w	Word character
\s	Space

Pattern Class

```
Pattern p =
```

```
Pattern.compile("[a-z]+");
```

Matcher Class

```
Matcher m =  
p.matcher("java");
```

Matching

```
System.out.println(m.matches());
```

Email Validation

```
String email = "abc@gmail.com";  
  
System.out.println(  
email.matches(  
"^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$"  
));
```

Q10. Explain JDBC in Detail

JDBC

JDBC stands for Java Database Connectivity.
Used to connect Java with database.

JDBC Architecture

Java Application



JDBC API



DriverManager



JDBC Driver



Database

Steps to Connect Database

Step 1 — Load Driver

```
Class.forName(  
    "com.mysql.cj.jdbc.Driver"  
);
```

Step 2 — Create Connection

```
Connection con =  
    DriverManager.getConnection(  
        "url","user","password"  
    );
```

Step 3 — Create Statement

```
Statement stmt =  
    con.createStatement();
```

Step 4 — Execute Query

```
ResultSet rs =  
stmt.executeQuery(  
"SELECT * FROM student"  
);
```

Step 5 — Process Result

```
while(rs.next()) {  
  
    System.out.println(  
        rs.getInt(1));  
}
```

Step 6 — Close Connection

```
con.close();
```

COMPLETE JDBC PROGRAM

```
import java.sql.*;  
  
class Demo {  
  
    public static void main(String args[]) {  
  
        try {  
  
            Class.forName(  
                "com.mysql.cj.jdbc.Driver");  
  
            Connection con =  
                DriverManager.getConnection(  
                    "jdbc:mysql://localhost:3306/test",  
                    "root",  
                    "password"  
                );
```

```

        Statement stmt =
con.createStatement();

ResultSet rs =
stmt.executeQuery(
"SELECT * FROM student"
);

while(rs.next()) {

    System.out.println(
rs.getInt(1));
}

con.close();

} catch(Exception e) {

    System.out.println(e);
}

}
}

```

PreparedStatement

Advantages

- Faster
- Secure
- Prevents SQL injection

Example

```

PreparedStatement ps =
con.prepareStatement(
"INSERT INTO student VALUES(?,?,?)"
);

ps.setInt(1,101);
ps.setString(2,"Robby");
ps.setInt(3,20);

```

```
ps.executeUpdate();
```

executeQuery vs executeUpdate

executeQuery()	executeUpdate()
Used for SELECT	Used for INSERT/UPDATE/DELETE
Returns ResultSet	Returns int

FINAL REVISION HACKS

MOST IMPORTANT LONG QUESTIONS

- OOP concepts
- Exception handling
- Multithreading
- Collections
- AWT + Event handling
- JDBC

MOST IMPORTANT PROGRAMS

- Inheritance
- Constructor overloading
- try-catch-finally
- Thread program
- ArrayList
- ActionEvent
- JDBC connection

NIGHT BEFORE EXAM

ONLY revise:

- JVM diagram

- Thread lifecycle
- JDBC steps
- Layout managers
- Regex symbols
- Differences tables